

Bitte beantworten Sie die Fragen in den dafür vorgesehenen Feldern. Wenn Sie keinen Platz mehr für eine Antwort haben, fahren Sie bitte auf der Rückseite fort und geben an wo es weiter geht.
Abgesehen von Internetfähigen Geräten sind alle Hilfsmittel zugelassen.
Schreiben Sie Ihre Klausur bitte nicht mit Bleistift und lassen Sie die Klausur bitte geheftet.
Sie haben 90 Minuten Zeit. Mit 43 Punkten gilt die Klausur als bestanden.

Vor- und Nachname (in Druckbuchstaben): _____

Matrikelnummer und Studiengang: _____

Aufgabe Nr.:	Punktzahl:	Davon erreicht:
1	6	
2	6	
3	11	
4	12	
5	12	
6	7	
7	7	
8	7	
9	9	
10	13	
Summe	90	

Note und Unterschrift des Erstprüfers: _____

Unterschrift des Zweitprüfers: _____

Lernraum I

1. (6 Punkte) Geben Sie die Laufzeit für die folgende Funktion f in der Θ -Notation an. Es sollte ersichtlich sein wie Sie auf diese Laufzeit gekommen sind (bspw. sehr kurze Begründung oder Gleichung).

```
def f(n):
    y = 0
    while (n >= 2):
        if n % 2 == 1:
            y = y + 1
        n = n/2

    return y
```

.....

.....

.....

.....

.....

.....

.....

.....

2. (6 Punkte) Geben Sie für jede der folgenden vier Funktionen die vereinfachte O-Notation in der zweiten Spalte der Tabelle an und beantworten Sie dann die entsprechende “Wahr oder Falsch” Frage zu dieser Funktion durch Einkreisen von “W” bzw. “F”.

Funktion	Vereinfachte O-Notation	Wahr (W) oder Falsch (W)? (Einkreisen)
$a(n) = 4\log_2(n)$		$a(n)$ ist in $O(n)$ W oder F
$b(n) = 4n + 3$		$b(n)$ ist in $\Omega(n \log n)$ W oder F
$c(n) = n^2 - 3n$		$c(n)$ ist in $O(n^2)$ W oder F
$d(n) = n^3 + 3n$		$d(n)$ ist in $\Omega(n^2)$ W oder F

Lernraum II

3. (11 Punkte) Ein Unternehmen möchte sicherstellen, dass seine Server regelmäßig gewartet werden. Es hat folgende Funktion `find_oldest_server` implementiert, die in einer for-Schleife alle Server durchgeht und den Server zurückgibt, der das älteste letzte Wartungsdatum hat.

Optimieren Sie die Funktion `find_oldest_server` durch die Speicherung der Server in einer geeigneten Datenstruktur und beschreiben Sie wie die neue Implementierung der Funktion `find_oldest_server` aussehen würde. Eine Beschreibung als Satz oder Pseudocode genügt. Python-Code ist alternativ auch möglich. Bestimmen Sie ebenfalls die Laufzeit der aktuellen Funktion `find_oldest_server` und die der neuen Implementierung in der O-Notation.

```
from datetime import datetime

class Server:
    def __init__(self, name, last_maintenance):
        self.name = name
        # Das Datum der letzten Wartung wird in dieser Variablen gespeichert
        self.last_maintenance = last_maintenance

    # ermöglicht Ausdrücke wie server1 < server2, wobei server1 und server2 Server Objekte sind
    def __lt__(self, other):
        return self.last_maintenance < other.last_maintenance

def find_oldest_server(servers):
    oldest_server = servers[0]
    for server in servers:
        if server.last_maintenance < oldest_server.last_maintenance:
            oldest_server = server
    return oldest_server

# Beispiel für die Nutzung: 3 Server mit zugehörigen Datumsangaben der letzten Wartung
servers = [
    Server("Server1", datetime(2023, 5, 1)),
    Server("Server2", datetime(2023, 3, 20)),
    Server("Server3", datetime(2023, 4, 15))
]

oldest = find_oldest_server(servers)
print(f"Der Server mit dem ältesten Wartungsdatum ist: {oldest.name}")
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

4. (12 Punkte) Ein soziales Netzwerk möchte Empfehlungen für neue Freunde basierend auf gemeinsamen Freunden geben. Entwickeln Sie einen Algorithmus in Pseudocode oder Wörtern, der einem Nutzer andere Nutzer zur Freundschaft vorschlägt. Dabei sollen solche Nutzer vorgeschlagen werden, die mit möglichst vielen Freunden des ursprünglichen Nutzers befreundet sind. In welcher Datenstruktur würden Sie die Anzahl gemeinsamer Freunde pro durchsuchten Nutzer speichern? Folgender Graph dient als Hilfestellung.

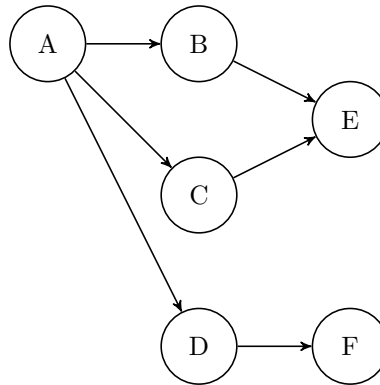


Abbildung 1: Der Knoten A ist der Nutzer dem neue Freunde vorgeschlagen werden sollen. Knoten B, C und D sind bereits Freunde von Knoten A. Der Algorithmus sollte Knoten E dem Nutzer A als neuen Freund vorschlagen, da E mit zwei Freunden von Knoten A befreundet ist. Knoten F soll Knoten A erst vorgeschlagen werden, wenn Knoten E bereits Knoten A vorgeschlagen wurde.

[illegible]

Lernraum III

5. (12 Punkte) Erweitern Sie den iterativen Algorithmus der Tiefensuche (DFS) so, dass er das Erreichbarkeitsproblem löst. Das Erreichbarkeitsproblem lautet: Gibt es bei einem Startknoten s und einem Zielknoten t einen Pfad zwischen s und t ? Ihr Algorithmus soll also `True` zurückliefern, wenn es einen Pfad gibt, sonst soll der Algorithmus `False` zurückliefern.

Der iterative Algorithmus der Tiefensuche lautet:

Algorithm 1 Iterative Tiefensuche (DFS)

```
1: function dfs(Graph graph, Vertex start) {
2:   Stack<Vertex> perimeter = new Stack<>();
3:   Set<Vertex> visited = new Set<>();
4:
5:   perimeter.add(start);
6:
7:   while (!perimeter.isEmpty()) {
8:     Vertex from = perimeter.remove();
9:     if (!visited.contains(from)) {
10:      for (Edge edge : graph.edgesFrom(from)) {
11:        Vertex to = edge.to();
12:        if (!visited.contains(to))
13:          perimeter.add(to);
14:      }
15:      visited.add(from);
16:    }
17:  }
18: }
```

Aufgabe: Geben Sie mit einer kurzen Begründung an, zwischen welchen Zeilen des Codes neuer Code hinzugefügt oder entfernt werden muss, um das Erreichbarkeitsproblem zu lösen. Code, den Sie hinzufügen kann Pseudocode sein. Es muss kein korrekter Java oder Python Code sein. Sie müssen nur die Zeilen angeben bei denen etwas geändert werden muss. Sie müssen nicht den vollständigen Code des gesamten Algorithmus abschreiben.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

6. Modellieren Sie ein Rezept zum Kuchen backen als Graph und geben Sie die Mindestdauer für den Gesamtvorgang an unter der Annahme, dass parallel durchführbare Vorgänge die Dauer der einzelnen Vorgänge nicht verlängert. Das Rezept hat die folgenden 9 Schritte und die Abhängigkeiten sind wie folgt gegeben.

1. Zutaten abwiegen: Muss als erstes gemacht werden, keine Abhängigkeiten. Dauer: 5 Minuten.
2. Zutaten mischen: Abhängig von “Zutaten abwiegen”. Dauer: 5 Minuten
3. Teig kneten: Abhängig von “Zutaten mischen”. Dauer: 10 Minuten
4. Backform vorbereiten: Kann unabhängig von den ersten Schritten durchgeführt werden. Dauer: 1 Minute
5. Teig in die Backform füllen: Abhängig von “Teig kneten” und “Backform vorbereiten”. Dauer: 1 Minute
6. Ofen vorheizen: Kann parallel zu anderen Schritten erfolgen, aber muss vor dem Backen abgeschlossen sein. Dauer: 15 Minuten
7. Kuchen backen: Abhängig von “Teig in die Backform füllen” und “Ofen vorheizen”. Dauer: 20 Minuten
8. Kuchen abkühlen lassen: Abhängig von “Kuchen backen”. Dauer: 30 Minuten.
9. Kuchen verzieren: Abhängig von “Kuchen abkühlen lassen”. Dauer: 8 Minuten.

(a) (6 Punkte) Graph:

A full-page sheet of white graph paper with a light gray grid. The grid consists of small squares, approximately 10 units wide by 10 units high. There are no margins or additional markings on the page.

(b) (1 Punkt) Gesamtdauer des Vorgangs:

.....

7. Angenommen, Sie sind Manager:in eines Telekommunikationsunternehmens und haben die Aufgabe, Glasfaserkabel zu verlegen, um mehrere Städte miteinander zu verbinden. Die Kosten für das Verlegen von Glasfaserkabeln zwischen den Städten sind unterschiedlich, und Ihr Ziel ist es, eine kostengünstige Netzwerkinfrastruktur zu schaffen.

(a) (3 Punkte) Welche Datenstruktur wählen Sie um das Problem zu modellieren? Wie modellieren Sie Städte, Glasfaserkabel und die Kosten für das Verlegen der Glasfaserkabel?

.....

.....

.....

.....

.....

.....

(b) (1 Punkt) Mit welchem Algorithmus bestimmen Sie die kostengünstigste Möglichkeit, die Städte über Glasfaser miteinander zu verbinden?

.....

.....

.....

.....

.....

.....

(c) (3 Punkte) Welchen Vor- und welchen Nachteil hat die Lösung, die durch den Algorithmus in (b) bestimmt wird?

.....

.....

.....

.....

.....

.....

8. (7 Punkte) Weighted Quick-Union mit Pfadkomprimierung: Rufen Sie `union(10, 25)` auf die folgenden beiden Bäume auf. Nutzen Sie dabei Weighted Quick-Union mit Pfadkomprimierung. Zeichnen Sie den sich ergebenden Baum. Hinweis: Es dürfte hilfreich sein Zwischenschritte zu zeichnen.

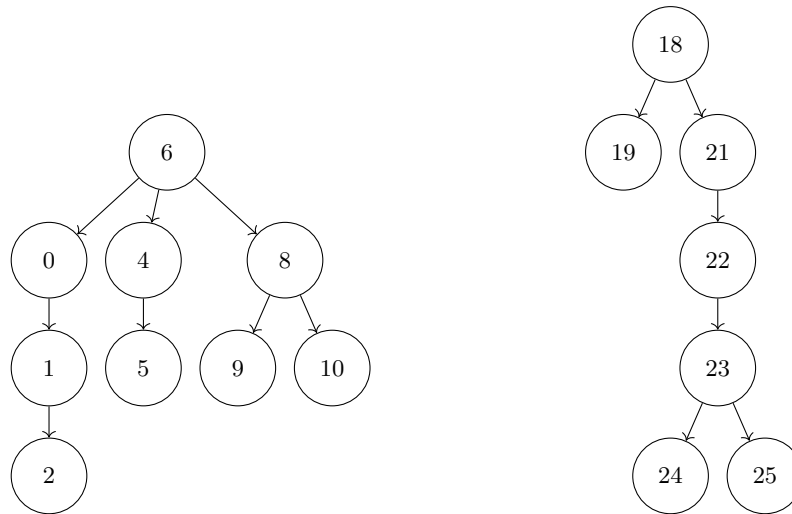
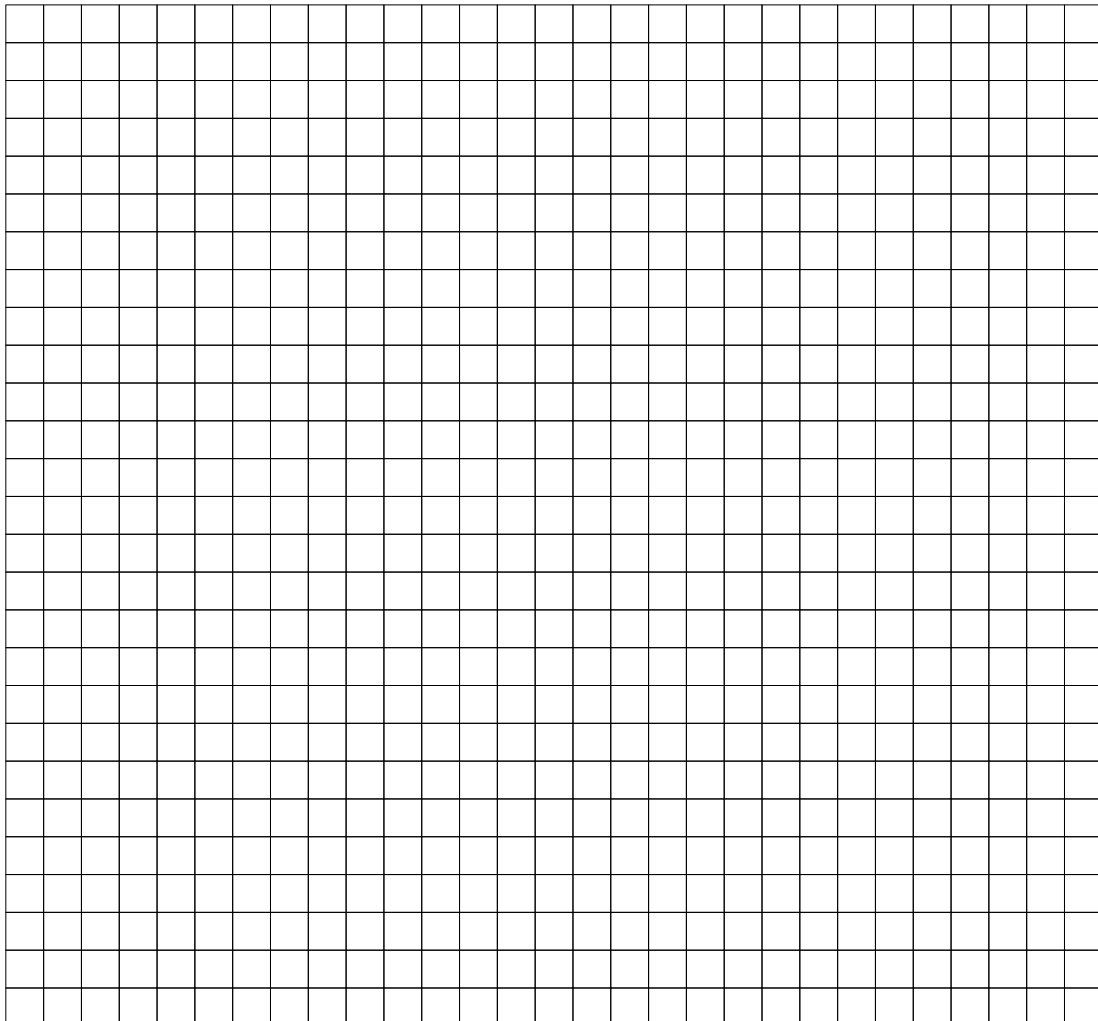


Abbildung 2: Darstellung von zwei Disjunkten Mengen auf denen Sie `union` aufrufen sollen.



Lernraum IV

9. Wählen Sie für jedes der folgenden Szenarien den passenden Sortieralgorithmus und begründen Sie Ihre Wahl in einem kurzen Satz.

Zur Wahl stehen: Insertionsort, Selectionsort, Mergesort, Quicksort, Heapsort, Bucketsort

- (a) (3 Punkte) Sie schreiben ein Programm, das die Bewerber für ein prestigeträchtiges Stipendium sortiert. Jeder Bewerbung wird eine numerische Punktzahl zugewiesen, um eine geordnete Rangfolge der Bewerbungen zu erstellen. Wenn zwei Bewerbungen die gleiche Punktzahl haben, wird die Bewerbung ausgewählt, die zuerst eingegangen ist.

.....

.....

.....

.....

.....

- (b) (3 Punkte) Sie sind eine Triage-Schwester und für die Warteschlange in der Notaufnahme einer Klinik zuständig, die bestimmt, wer zuerst zum Arzt geht. Sie führen eine einzige sortierte Liste in einer Tabellenkalkulation, und wenn neue Patienten eintreffen, fügen Sie sie entsprechend dem Schweregrad ihrer Verletzung in die Warteschlange ein. Wenn zwei Patienten den gleichen Schweregrad haben, sollte der Patient, der zuerst eintrifft, den Arzt zuerst sehen.

.....

.....

.....

.....

.....

- (c) (3 Punkte) Ein Softwareunternehmen möchte die Performance seiner Mitarbeiter für ein bestimmtes Projekt evaluieren. Die Performance-Werte der Mitarbeiter liegen zwischen 0 und 10 (Ganzzahlen), wobei es sehr viele Mitarbeiter gibt. Sie möchten die Performance-Werte effizient sortieren, um die besten und schlechtesten Leistungen leicht identifizieren zu können.

.....

.....

.....

.....

.....

10. (13 Punkte) Das Partitionsproblem besteht darin, zu bestimmen, ob eine gegebene Menge von Ganzzahlen in zwei Teilmengen aufgeteilt werden kann, so dass die Summe der Elemente in beiden Teilmengen gleich ist.

Bspw. kann die Liste [3, 4, 1] in [3, 1] und [4] partitioniert werden, da die Summe der Elemente in beiden Listen 4 beträgt.

Die Brute-Force Lösung lautet:

```
def isSubsetSum(arr, n, sum):
    if sum == 0:
        return True
    if n == 0 and sum != 0:
        return False

    if arr[n-1] > sum:
        return isSubsetSum(arr, n-1, sum)

    return isSubsetSum(arr, n-1, sum) or isSubsetSum(arr, n-1, sum-arr[n-1])

def findPartion(arr, n):
    sum = 0
    for i in range(0, n):
        sum += arr[i]

    if sum % 2 != 0:
        return false

    return isSubsetSum(arr, n, sum // 2)

if __name__ == '__main__':
    arr = [3, 4, 1]
    n = len(arr)

    if findPartion(arr, n) == True:
        print("Can be divided into two subsets of equal sum")
    else:
        print("Can not be divided into two subsets of equal sum")
```

Zeichnen Sie dazu den Rekursionsbaum der Methode `isSubsetSum` für das im Code aufgerufene Beispiel. Damit Sie nicht so viel schreiben müssen, würde ich Ihnen empfehlen in die Knoten nur die beiden Argumente `n` und `sum` einzuzeichnen. Der Wurzelknoten des Rekursionsbaums sieht also so aus:



Rechenkästchen folgen auf der nächsten Seite.

